Guardian Shell For Product Managers

Prepared for product review Date: March 13, 2026 Audience: Product managers, business stakeholders, delivery leads

---

Executive Summary

Guardian Shell is a Linux security product designed for AI agents such as coding assistants and autonomous CLI tools.

Its purpose is to answer a business question that normal Linux controls do not answer well:

"What can this specific AI agent session access, and how do we govern it safely?"

The product matters because AI agents are not ordinary software processes. They:

- use many tools and subprocesses
- adapt when blocked
- may search for workarounds
- need temporary, auditable approvals in real workflows

Guardian Shell is strong because it combines:

1. agent-level identity
2. kernel-level monitoring and blocking
3. human approval workflow
4. dashboard, alerts, and auditability

The biggest current weakness is that too much of the policy still depends on path-based decisions and tracepoint-first logic.

The recommended direction is to keep the product surface, while strengthening the kernel trust anchor underneath it.

The Core Problem

A developer may allow an AI agent to work on one project folder, but the agent often runs with the same operating-system access as the developer.

That means the agent may also be able to read:

- SSH keys

- cloud credentials

- .env files

- browser tokens

- internal company documents

Traditional Linux permissions usually control what a user can access. Guardian Shell is designed to control what one AI agent session can access.

Why eBPF Is Useful For LLM Agents

This is the key decision question for non-engineering stakeholders.

Why not just use chmod, chown, ACLs, or sticky bit?

Those controls are useful baseline hygiene, but they do not naturally solve:

- one agent session versus another

- dynamic approvals

- subprocess governance

- audit trails tied to a live agent task

Why not just use AppArmor or SELinux?

Both are valuable baseline Linux security systems.

- AppArmor is easier and path-based

- SELinux is stronger and label-based

But neither is naturally a PM-friendly agent-governance workflow product.

Guardian Shell adds:

- per-agent session thinking

- runtime-updated policy

- approval workflow

- user-friendly visibility

The practical answer

The best answer is not:

"Use eBPF instead of Linux security."

The better answer is:

"Use Guardian Shell’s eBPF-based control plane for AI-agent governance, and layer it with baseline Linux controls where appropriate."

What Guardian Shell Does Today

The current repository shows that Guardian Shell already supports:

- file monitoring and some file blocking
- command monitoring and some exec blocking
- outbound network monitoring
- per-agent identity using cgroups, TGIDs, and process names
- a launcher for cgroup-based isolation
- dashboards, alerts, and permission requests

Why this is valuable

This means Guardian Shell is already more than a kernel probe. It is becoming a usable governance product.

It addresses business needs such as:

- least privilege for agents
- auditability
- temporary approvals
- operational visibility

How Guardian Shell Identifies An Agent

This is one of the most important strengths of the design.

AI agents do not stay as one process. They often spawn:

- shells
- interpreters
- package managers
- test runners
- git

Guardian Shell therefore uses a layered identity model:

1. best: cgroup identity
2. fallback: TGID / process-tree tracking
3. fallback: process name (comm)

Why cgroup identity matters

When an agent is launched through guardian-launch, it is placed in a dedicated cgroup.

That means:

- the main process is identified
- child processes stay in the same group
- the whole session can be governed as one unit

This is much better than relying only on process names like python3 or node.

Strengths Of The Current Product

1. Built around the real unit of control

The product thinks in terms of agent sessions rather than just binaries.

2. Human approval workflow

A manager or operator can understand:

"The agent requested access to this folder for 15 minutes."

This is much easier to operate than manually editing Linux policy.

3. Good operational surface

The current product already includes:

- dashboard
- alerts
- CLI controls
- metrics
- structured event handling

4. Better fit for coding-agent workflows

Coding agents are interactive and tool-heavy.

Guardian Shell is better suited to this environment than static, program-only policy models.

Where The Current Product Is Weak

1. Path-based policy is still the main weakness

If policy depends too much on a path string, an adaptive agent may try alternate paths such as:

- /proc/self/root/...
- symlink paths
- bind-mount aliases
- hard-link aliases

2. Two-step enforcement is harder to trust

Today, Guardian often uses:

1. tracepoint decides
2. LSM hook blocks later

That works, but it is harder to reason about than making the decision directly at the final security hook.

3. Exec control is weaker than binary-identity systems

Path-based exec control is weaker than binary-hash identity.

This matters when an agent copies or renames a tool to bypass policy.

4. Network control is not yet fully mature

The current product is better at network visibility than precise policy-grade egress control.

Guardian Shell vs Veto

Guardian Shell and Veto are not the same kind of product.

Veto is better at:

- strong binary identity
- pre-execution blocking

- resisting rename/copy tricks for executables

Guardian Shell is better at:

- file governance
- per-agent policy
- subprocess-aware session governance
- temporary approvals
- dashboard and workflow features

The clearest product message

- Veto answers: "Should this binary run?"
- Guardian Shell answers: "What can this agent session access?"

They are complementary more than directly competing.

Recommended Product Direction

The product should not throw away its current strengths.

The right path is:

1. keep the current dashboard, CLI, approval, and policy workflow model
2. make cgroup-backed identity the default secure mode
3. move core file and exec decisions deeper into LSM-based enforcement
4. add seccomp as a standard hardening layer
5. offer stronger isolation tiers for high-risk agent deployments

Suggested positioning

Guardian Shell should be positioned as:

"A Linux agent-governance platform with kernel visibility, policy enforcement, and human approval workflows."

That is a stronger and more accurate message than positioning it only as a blocking engine.

Final Takeaway

Guardian Shell is promising because it solves a broader and more operationally useful

problem than many low-level security tools.

Its product advantage is not only kernel enforcement. Its advantage is the combination of:

- agent identity
- governance
- approvals
- visibility
- workflow integration

If the kernel trust anchor is strengthened over time, the product can occupy a very useful position between:

- basic Linux permissions
- static host MAC systems
- heavyweight isolated runtimes

References

Internal material used:

- docs/architecture-analysis.md
- docs/guardian-shell-vs-veto-comparison.md
- docs/security-improvements-research.md
- docs/sandboxing-deep-dive.md
- AGENT_IDENTITY.md
- implementation in guardian-ebpf/src/main.rs, guardian/src/main.rs, and guardian-launch/src/main.rs

External references checked:

- Ona, "Introducing Veto: security for the next era of software"

https://ona.com/stories/introducing-veto-security-for-the-next-era-of-software

- Linux kernel docs, "LSM BPF Programs"

https://docs.kernel.org/bpf/prog_lsm.html

- Ubuntu documentation, "AppArmor"

https://ubuntu.com/server/docs/how-to/security/apparmor/

- Red Hat documentation, "Using SELinux"
https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html-single/using_selinux/using_selinux